INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	**INTRODUCCIÓN AL DESARROLLO WEB**				
TÍTULO DE LA PRÁCTICA:	*Entorno de Ejecución Node.js, Gestión de Paquetes con npm y Creación de Servidor HTTP con Módulos Nativos*				
NÚMERO DE PRÁCTICA:	*05*	**AÑO LECTIVO:**	*2026*	**NRO. SEMESTRE:**	*II*
FECHA DE PRESENTACIÓN	*19/10/2026*	**HORA DE PRESENTACIÓN**	*hh/mm/ss*		
INTEGRANTE (s) - *Gomez Huaman, Melany Yaritza* - *Morales Yapu, Hillary Fernanda* - *Ortiz Saavedra, Sarah Megan* - *Paredes Gonzales, Melanie Grace*				**NOTA (0-20)**	
DOCENTE(s): MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.					

RESULTADOS Y PRUEBAS

I. EJERCICIOS RESUELTOS:

Servidor HTTP básico resuelto utilizando módulos nativos:

```
// server.js
const http = require('http');
const fs = require('fs');
const path = require('path');

const PORT = process.env.PORT || 3000;

const sendJSON = (res, status, body) => {
  res.writeHead(status, { 'Content-Type': 'application/json; charset=utf-8'
});
  res.end(JSON.stringify(body));
};

const server = http.createServer((req, res) => {
  const { pathname } = new URL(req.url, `http://${req.headers.host}`);
  console.log(`Petición recibida: ${req.method} ${pathname}`);
```

```
  if (req.method === 'GET' && pathname === '/') {
    const filePath = path.join(__dirname, 'public', 'index.html');
    return fs.readFile(filePath, (err, content) => {
      if (err) {
        const status = err.code === 'ENOENT' ? 404 : 500;
        return sendJSON(res, status, {
          message: status === 404 ? 'Recurso no encontrado' : 'Error interno
del servidor',
        });
      }
      res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
      res.end(content);
    });
  }

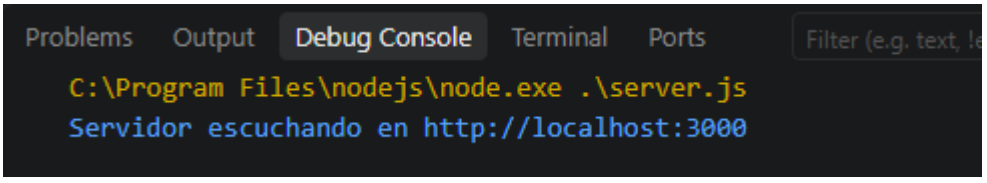
  if (req.method === 'GET' && pathname === '/api/status') {
    return sendJSON(res, 200, { status: 'OK', uptime: process.uptime() });
  }

  sendJSON(res, 404, { message: 'Recurso no encontrado' });
});

server.listen(PORT, () => {
  console.log(`Servidor escuchando en http://localhost:${PORT}`);
});

process.on('SIGINT', () => server.close(() => process.exit(0)));
process.on('SIGTERM', () => server.close(() => process.exit(0)));
```

Ejecución:



II. PROBLEMAS PROPUESTOS RESUELTOS:

Caso Práctico: Servidor Backend Estático y API Básica de Consultas Desarrollar una aplicación en Node.js desde cero implementando los siguientes requerimientos:

1. Configuración de Proyecto npm:

- Inicializar el proyecto ejecutando npm init -y.
- Configurar scripts en el package.json: "start": "node server.js" y "dev": "node --watch server.js".
- Crear una estructura modular de carpetas: /public (archivos estáticos HTML/CSS), /data (archivo JSON con datos) y server.js.

2. Construcción del Servidor HTTP Nativo:

- Instanciar el servidor mediante http.createServer escuchando en el puerto 3000.
- Servir archivos estáticos desde la carpeta /public asíncronamente con fs.readFile infiriendo correctamente el Content-Type (ej. text/html, text/css).

3. Implementación de Endpoints Basados en Rutas:

- Ruta GET /api/estudiantes: Leer el archivo /data/estudiantes.json de forma asíncrona y retornar la lista de estudiantes en formato JSON con código 200 OK.
- Ruta POST /api/estudiantes: Capturar los chunks de datos del body de la petición (evento req.on('data') y req.on('end')), parsear el objeto, agregarlo al archivo JSON de persistencia local y retornar el registro guardado con código 201 Created.
- Manejo de rutas no existentes retornando un JSON con código 404 Not Found

- **package.json**

```
{
  "name": "backend-estudiantes-node",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "node server.js",
    "dev": "node --watch server.js"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/Melanie-pg-bit/backend-estudiantes-node.git"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "bugs": {
    "url": "https://github.com/Melanie-pg-bit/backend-estudiantes-node/issues"
```

```
  },
  "homepage": "https://github.com/Melanie-pg-bit/backend-estudiantes-node#readme"
}
```

- **data/estudiantes.json**

```
[
  {
    "id": 1,
    "nombre": "Juan"
  },
  {
    "id": 2,
    "nombre": "María"
  },
  {
    "id": 3,
    "nombre": "Carlos"
  },
  {
    "id": 1792383379985,
    "nombre": "Rosa Zegarra"
  }
]
```

- **public/index.html**

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Servidor de Estudiantes</title>
    <link rel="stylesheet" href="/styles.css">
</head>
<body>

    <h1>Servidor de Estudiantes</h1>

    <div class="box">
        <h3>Registrar Estudiante</h3>
        <form id="formEstudiante">
            <input type="text" id="nombreInput" placeholder="Nombre del estudiante"
required>
            <button type="submit">Guardar</button>
        </form>
    </div>
```

```
    <div class="box">
        <h3>Lista de Estudiantes</h3>
        <ul id="listaEstudiantes"></ul>
    </div>

   <script>
    async function cargarEstudiantes() {
        try {
            const res = await fetch('/api/estudiantes');
            const estudiantes = await res.json();
            const lista = document.getElementById('listaEstudiantes');
            lista.innerHTML = '';

            if (!Array.isArray(estudiantes) || estudiantes.length === 0) {
                lista.innerHTML = '<li>No hay estudiantes aún</li>';
                return;
            }

            estudiantes.forEach(est => {
                const li = document.createElement('li');
                li.textContent = typeof est === 'object' ? (est.nombre ||
JSON.stringify(est)) : est;
                lista.appendChild(li);
            });
        } catch (err) {
            console.error("Error al obtener la lista:", err);
        }
    }

    document.getElementById('formEstudiante').addEventListener('submit', async (e) =>
{
        e.preventDefault();
        const input = document.getElementById('nombreInput');
        const nombreVal = input.value.trim();

        if (!nombreVal) return;

        try {
            const respuesta = await fetch('/api/estudiantes', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ nombre: nombreVal })
            });
```

```
            const resultado = await respuesta.json();

            if (respuesta.ok) {
                input.value = '';
                cargarEstudiantes();
            } else {
                // Muestra exactamente el motivo del error en una ventana flotante
                alert("ERROR DEL SERVIDOR: " + (resultado.detalle ||
JSON.stringify(resultado)));
            }
        } catch (err) {
            alert("Error de conexión: " + err.message);
        }
    });

    cargarEstudiantes();
</script>
</body>
</html>
```

- **public/style.css**

```
body {
    font-family: Arial, sans-serif;
    background-color: #f9f9f9;
    color: #333;
    margin: 40px auto;
    max-width: 500px;
    padding: 20px;
}

h1 {
    color: #2c3e50;
    text-align: center;
    margin-bottom: 20px;
}

.box {
    background: white;
    padding: 20px;
    border-radius: 8px;
    border: 1px solid #ddd;
    margin-bottom: 20px;
}
```

```
form {
    display: flex;
    gap: 10px;
}

input {
    flex: 1;
    padding: 8px;
    border: 1px solid #ccc;
    border-radius: 4px;
}

button {
    background: #007bff;
    color: white;
    border: none;
    padding: 8px 15px;
    border-radius: 4px;
    cursor: pointer;
}

button:hover {
    background: #0056b3;
}

ul {
    padding-left: 20px;
}

li {
    margin-bottom: 8px;
}
```

- **staticHandler.js**

```
const fs = require('fs');
const path = require('path');

function manejarEstaticos(req, res) {
    // Si la ruta es "/" le asignamos "/index.html" por defecto
    let url = req.url === '/' ? '/index.html' : req.url;
    const rutaArchivo = path.join(__dirname, 'public', url);
```

```
    // Determinar la extensión para el Content-Type
    const ext = path.extname(rutaArchivo);
    let contentType = 'text/html';

    if (ext === '.css') contentType = 'text/css';
    if (ext === '.js') contentType = 'application/javascript';
    if (ext === '.json') contentType = 'application/json';

    fs.readFile(rutaArchivo, (err, data) => {
        if (err) {
            res.writeHead(404, { 'Content-Type': 'application/json' });
            res.end(JSON.stringify({ mensaje: '404 Not Found' }));
            return;
        }

        res.writeHead(200, { 'Content-Type': contentType });
        res.end(data);
    });
}

module.exports = { manejarEstaticos };
```

- **getEstudiantes.js**

```
const fs = require('fs');
const path = require('path');

function obtenerEstudiantes(req, res) {

    const rutaArchivo = path.join(__dirname, 'data', 'estudiantes.json');

    fs.readFile(rutaArchivo, 'utf8', (err, data) => {
        if (err) {
            res.writeHead(404, { 'Content-Type': 'application/json' });
            res.end(JSON.stringify({ mensaje: 'Error al leer la base de datos' }));
            return;
        }
        res.writeHead(200, { 'Content-Type': 'application/json' });
        res.end(data);
    });
}

module.exports = { obtenerEstudiantes };
```

- **postEstudiantes.js**

```
const fs = require('fs');
const path = require('path');

function guardarEstudiante(req, res) {
    let body = '';

    req.on('data', chunk => {
        body += chunk.toString();
    });

    req.on('end', () => {
        try {
            const dataParseada = JSON.parse(body);
            const rutaArchivo = path.join(__dirname, 'data', 'estudiantes.json');

            fs.readFile(rutaArchivo, 'utf8', (err, data) => {
                let estudiantes = [];

                if (err) {
                    // Muestra en pantalla si no encuentra la carpeta o el archivo
                    res.writeHead(500, { 'Content-Type': 'application/json' });
                    res.end(JSON.stringify({ detalle: 'Error al leer estudiantes.json:
' + err.message }));
                    return;
                }

                try {
                    estudiantes = JSON.parse(data);
                } catch (e) {
                    res.writeHead(500, { 'Content-Type': 'application/json' });
                    res.end(JSON.stringify({ detalle: 'El archivo estudiantes.json
tiene un error de formato JSON' }));
                    return;
                }

                const nuevoEstudiante = {
                    id: Date.now(),
                    nombre: dataParseada.nombre || 'Sin Nombre'
                };

                estudiantes.push(nuevoEstudiante);
```

```
                fs.writeFile(rutaArchivo, JSON.stringify(estudiantes, null, 2),
(errWrite) => {
                    if (errWrite) {
                        res.writeHead(500, { 'Content-Type': 'application/json' });
                        res.end(JSON.stringify({ detalle: 'Error al escribir
estudiantes.json: ' + errWrite.message }));
                        return;
                    }

                    res.writeHead(201, { 'Content-Type': 'application/json' });
                    res.end(JSON.stringify(nuevoEstudiante));
                });
            });
        } catch (error) {
            res.writeHead(400, { 'Content-Type': 'application/json' });
            res.end(JSON.stringify({ detalle: 'El JSON recibido no es válido: ' +
error.message }));
        }
    });
}

module.exports = { guardarEstudiante };
```

- **server.js:**

```
const http = require('http');
const { obtenerEstudiantes } = require('./getEstudiantes');
const { guardarEstudiante } = require('./postEstudiantes');
const { manejarEstaticos } = require('./staticHandler');

const PORT = 3000;

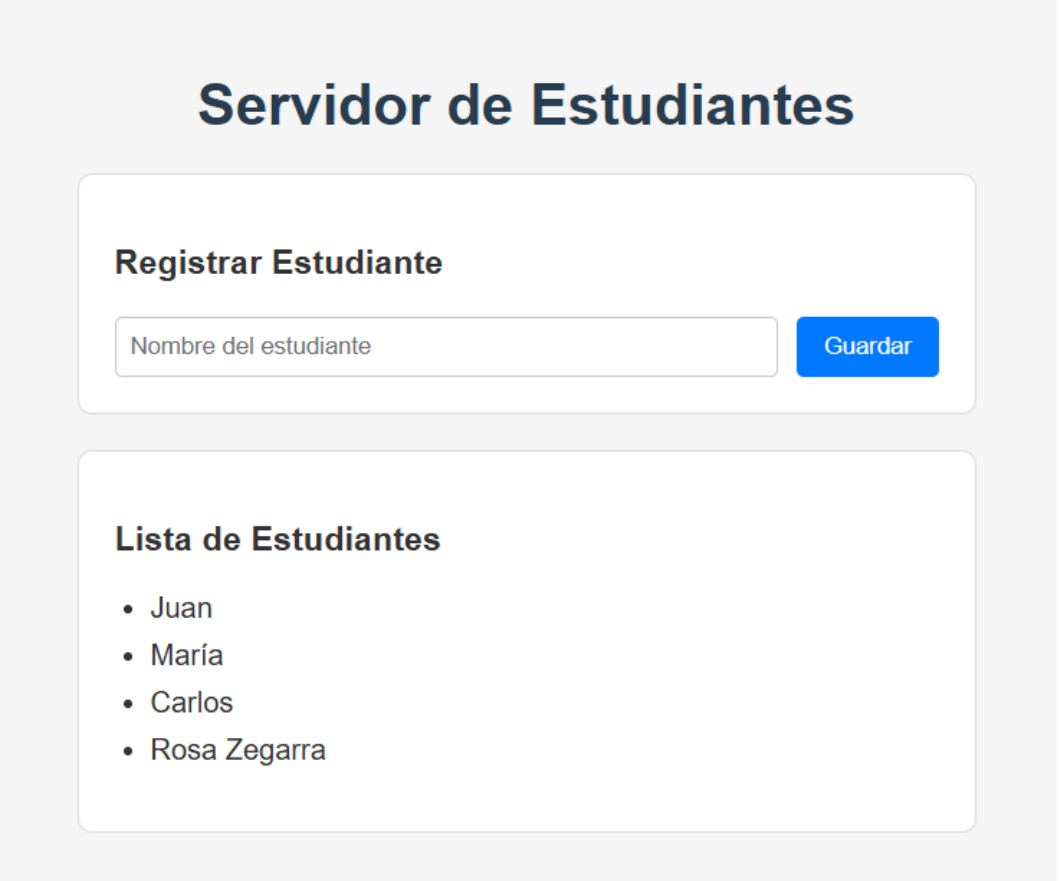
const server = http.createServer((req, res) => {
    if (req.url === '/api/estudiantes' && req.method === 'GET') {
        return obtenerEstudiantes(req, res);
    }

    if (req.url === '/api/estudiantes' && req.method === 'POST') {
        return guardarEstudiante(req, res);
    }

    // Cualquier otra ruta pasa al manejador de archivos de la carpeta public
    manejarEstaticos(req, res);
});
```

```
server.listen(PORT, () => {
    console.log(`Servidor ejecutándose en http://localhost:${PORT}`);
});
```

- **Ejecución:**



REPOSITORIO GITHUB: https://github.com/Melanie-pg-bit/backend-estudiantes-node

III. CUESTIONARIO:

1. **Explique la arquitectura orientada a eventos y el modelo de I/O No Bloqueante (Non-blocking I/O) de Node.js. ¿En qué se diferencia de servidores multihilo tradicionales como Apache Tomcat?**

   Node.js usa un solo hilo. Cuando lee archivos o bases de datos, no se congela a esperar; delega la tarea y sigue atendiendo a otros usuarios. Al terminar, el bucle de eventos envía la respuesta. Servidores como Apache Tomcat asignan un hilo por usuario, lo que puede saturar

la memoria si entran muchas personas a la vez. Node.js evita esto al procesar todo en un solo hilo de forma rápida y continua.

2. **¿Qué función cumple la carpeta node_modules y por qué nunca debe incluirse en el control de versiones (Git)? ¿Qué archivo controla exactamente el árbol de dependencias?**

   Node_modules guarda las librerías del proyecto. No se sube a Git porque es muy pesada y no es necesario guardar miles de archivos en el repositorio. El archivo que controla las versiones exactas de las librerías es package-lock.json. Con él, cualquiera en el equipo puede descargar lo mismo usando npm install.

3. **¿Cuál es la diferencia entre utilizar métodos síncronos de File System (fs.readFileSync) frente a métodos asíncronos (fs.readFile) en un servidor de producción?**

   El método síncrono (fs.readFileSync) congela todo el servidor mientras lee un archivo, bloqueando a los demás usuarios. Solo se usa al encender la app para las configuraciones iniciales. El método asíncrono (fs.readFile) lee en segundo plano sin pausar nada. El servidor sigue atendiendo a los usuarios sin interrupciones, siendo la opción correcta en producción.

CONCLUSIONES

- La arquitectura asíncrona de Node.js es fundamental para crear servidores web ligeros y rápidos, ya que al no bloquear el hilo de ejecución mientras se leen o guardan datos en el archivo JSON, la aplicación sigue respondiendo a otros usuarios sin quedarse congelada.
- El uso de un servidor modular facilita mucho el mantenimiento del código, permitiendo separar de forma clara la lógica de la API (para consultar y registrar estudiantes) del manejo de archivos estáticos como la interfaz HTML y los estilos CSS.
- La persistencia de datos mediante archivos JSON combinada con funciones de File System es una alternativa sencilla e ideal para entender cómo viajan, se procesan y se almacenan los datos en una aplicación backend antes de pasar a bases de datos tradicionales.
- Mantener buenas prácticas de desarrollo, como ocultar la carpeta node_modules en Git mediante .gitignore y estructurar correctamente las rutas HTTP, garantiza un proyecto limpio, liviano y listo para trabajar colaborativamente.

METODOLOGÍA DE TRABAJO

El proyecto se trabajó de forma modular en Node.js creando un servidor HTTP que atiende peticiones API y entrega archivos estáticos. Los datos se procesan asíncronamente con fs para leer y guardar

estudiantes en un archivo JSON sin pausar el sistema, todo respaldado con Git para el control de versiones.

REFERENCIAS Y BIBLIOGRAFÍA

- *Node.js Docs. (2025). Node.js v22.x API Reference Documentation: HTTP & FS Modules. Nodejs.org.*
- *Cantelon, M., Harter, M., Holowaychuk, T., & Rajlich, N. (2017). Node.js in Action (2nd ed.). Manning Publications*